

Comprehensive Study Guide: Python Programming & OOP

Prepared for: Abhishek Shinde

Topics Covered: Core Python Programming, Object-Oriented Programming (OOP), and Advanced Python Concepts.

Part I: Core Python Programming & Data Structures

Python is a versatile, high-level programming language widely used in software development, data analysis, and automation. Understanding how Python stores and manipulates data in memory is the first step to mastering the language.

1. The Concept of Mutability

In Python, every piece of data is an object. The type of object determines whether its internal state or contents can be changed after it is created in the system's memory.

Immutable Data Types

Immutable objects **cannot** be modified after their creation. If you attempt to alter an immutable object, Python does not change the original; instead, it creates a completely new object in memory and updates the variable reference.

- **Numeric Types (int, float, complex):** Used for all mathematical operations.
- **Strings (str):** Text data enclosed in single, double, or triple quotes. While strings support methods like `.upper()` or slicing, these methods return *new* strings rather than altering the original data.
- **Tuples (tuple):** Ordered, unchangeable collections defined by parentheses `()`. Because they cannot be changed, tuples are faster and more memory-efficient than lists. They are ideal for fixed datasets like geographic coordinates or RGB color codes.

Mutable Data Types

Mutable objects **can** be modified in place. Their memory address remains exactly the same even as you add, remove, or change their contents.

- **Lists (list):** Ordered, dynamic arrays defined by square brackets `[]`. They can hold mixed data types and allow duplicate values. You can change a list using methods like `.append()`, `.remove()`, or direct index assignment.
- **Dictionaries (dict):** Data structures that store information in key-value pairs, defined by curly braces `{}`. Keys must be strictly immutable (like strings or numbers), while values can

be anything. They are highly optimized for fast data retrieval.

- **Sets (set):** Unordered collections of unique elements. They are incredibly useful for mathematical operations (unions, intersections) and for instantly stripping duplicate values out of a list.

Quick Reference Table: Data Types

Category	Data Type	Mutable?	Syntax Example
Text	str	No	name = "Abhishek"
Numeric	int, float	No	age = 25, pi = 3.14
Sequence	list	Yes	scores = [90, 85, 88]
Sequence	tuple	No	location = (12.9, 77.5)
Mapping	dict	Yes	user = {'id': 1, 'role': 'admin'}
Set	set	Yes	unique_ids = {101, 102, 103}

Part II: Object-Oriented Programming (OOP) in Python

OOP is a programming paradigm that organizes software design around data (objects) rather than just functions and logic. A **Class** acts as a blueprint, and an **Object** is a specific instance created from that blueprint.

1. Encapsulation

Encapsulation bundles data (attributes) and the methods (functions) that manipulate that data into a single, cohesive unit (the class). More importantly, it restricts direct outside access to specific components to protect the integrity of the data.

- **Public Attributes:** Accessible from anywhere in the code.
- **Protected Attributes:** Denoted by a single underscore (e.g., `_name`). This is a developer convention indicating the attribute is intended for internal use and subclasses only.

- **Private Attributes:** Denoted by a double underscore (e.g., `__balance`). This strictly hides the attribute from the outside world using a technique called "name mangling".

Note: To safely interact with private variables, developers use public methods known as **Getters** (to view the data) and **Setters** (to modify the data securely).

2. Abstraction

Abstraction hides complex, underlying implementation details and exposes only the necessary, high-level features to the user. In Python, this is primarily achieved using the `abc` (Abstract Base Classes) module. An abstract class defines a template that child classes *must* follow, but it cannot be instantiated on its own.

3. Inheritance

Inheritance allows a new class (the Child or Derived class) to inherit attributes and methods from an existing class (the Parent or Base class). This heavily promotes code reusability and logical hierarchy.

- **Single Inheritance:** One child inherits from one parent.
- **Multiple Inheritance:** A child inherits from multiple parents simultaneously (Python fully supports this, unlike some other languages).
- **Multilevel Inheritance:** A continuous chain of inheritance (Grandparent $\rightarrow$ Parent $\rightarrow$ Child).

4. Polymorphism

Derived from Greek meaning "many forms," polymorphism allows objects of different classes to be treated as objects of a common superclass. The most common implementation in Python OOP is **Method Overriding**, where a child class provides a customized implementation of a method that is already defined in its parent class.

Part III: Advanced Python Concepts

Once the fundamentals are mastered, Python offers powerful advanced features for writing cleaner, safer, and more efficient code.

1. File Handling & Data Persistence

To save data permanently, Python uses the built-in `open()` function.

- **Modes:** 'r' (read), 'w' (write/overwrite), 'a' (append to the end of a file).
- **The with Statement:** This is the industry-standard best practice for file operations. It acts as a context manager that automatically closes the file once the block of code finishes executing, even if an unexpected error occurs.

2. Exception Handling

Robust software must handle runtime errors gracefully without crashing the entire application.

- try: The block containing code that might potentially cause an error.
- except: The block that runs if (and only if) an error occurs in the try block.
- else: The block that runs if the try block succeeds without any errors.
- finally: The cleanup block that executes no matter what happens (often used to close database connections).

3. Decorators (@)

A decorator is a powerful tool that allows you to modify or enhance the behavior of a function or method without permanently altering its source code. They act as "wrappers" and are heavily used in modern frameworks for tasks like user authentication, route definition, and logging.

4. Lambda Functions

Lambdas are anonymous, single-line functions used for short, throwaway operations where defining a full standard function (using def) would be overkill.

5. List Comprehensions

List comprehensions provide a highly readable, elegant, and Pythonic way to generate new lists based on existing iterables. They combine a for loop and an optional if condition into a single line of code, often executing faster than traditional loops.

railway-analysis-submission

February 19, 2026

1 Complete Indian Railway Reservation Analysis

This notebook performs a comprehensive analysis of the `indian-railway-reservation-tiny-dataset.csv` dataset. It covers data loading, cleaning, missing value handling, exploratory data analysis (EDA), and visualization of key metrics using strictly Matplotlib.

1.1 Table of Contents

1. Imports and Setup
2. Data Loading and Inspection
3. Data Cleaning and Missing Value Handling
4. Feature Engineering
5. Exploratory Data Analysis (EDA)
 - Univariate Analysis
 - Bivariate Analysis
 - Time Series Analysis
6. Conclusion

1.2 Importing all the important libraries

```
[30]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
plt.rcParams['figure.figsize'] = (10, 6)
```

1.3 Data Loading and Inspection

1.4 Load the dataset

```
[3]: df = pd.read_csv(r"C:\Users\abhishek\OneDrive\Desktop\indian-railway-reservation-tiny-dataset.\
↵csv")
```

1.5 Display the first 5 rows

```
[4]: display(df.head())
```

	Unnamed: 0	primary_key	Train No	Arrival time	Departure Time	\
0	0	0	107	00:00:00	10:25:00	
1	1	1	107	11:06:00	11:08:00	
2	2	2	107	11:28:00	11:30:00	
3	3	3	107	12:10:00	00:00:00	
4	4	4	108	00:00:00	20:30:00	

	Source Station	Destination Station	Available Seats	dirty_bit
0	SWV	MAO	1000	0
1	SWV	MAO	1000	0
2	SWV	MAO	1000	0
3	SWV	MAO	1000	0
4	MAO	SWV	1000	0

1.6 Display the last 5 rows

```
[5]: display(df.tail())
```

	Unnamed: 0	primary_key	Train No	Arrival time	Departure Time	\
9995	9995	9999	12146	12:30:00	12:55:00	
9996	9996	10000	12146	13:45:00	13:50:00	
9997	9997	10001	12146	15:38:00	15:40:00	
9998	9998	10002	12146	17:50:00	18:00:00	
9999	9999	10003	12146	20:42:00	20:45:00	

	Source Station	Destination Station	Available Seats	dirty_bit
9995	PURI	LTT	1000	0
9996	PURI	LTT	1000	0
9997	PURI	LTT	1000	0
9998	PURI	LTT	1000	0
9999	PURI	LTT	1000	0

1.7 Display the information about the dataset

```
[6]: print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            10000 non-null  int64
1   primary_key           10000 non-null  int64
2   Train No              10000 non-null  int64
```

```

3   Arrival time          10000 non-null object
4   Departure Time        10000 non-null object
5   Source Station         10000 non-null object
6   Destination Station    10000 non-null object
7   Available Seats        10000 non-null int64
8   dirty_bit              10000 non-null int64
dtypes: int64(5), object(4)
memory usage: 703.3+ KB
None

```

1.8 Display statistics summary about the dataset

```
[7]: display(df.describe())
```

	Unnamed: 0	primary_key	Train No	Available Seats	dirty_bit
count	10000.00000	10000.000000	10000.000000	10000.0	10000.0
mean	4999.50000	5002.574800	9040.921500	1000.0	0.0
std	2886.89568	2888.127806	3057.040038	0.0	0.0
min	0.00000	0.000000	107.000000	1000.0	0.0
25%	2499.75000	2503.750000	7044.000000	1000.0	0.0
50%	4999.50000	5003.500000	11021.000000	1000.0	0.0
75%	7499.25000	7503.250000	11204.000000	1000.0	0.0
max	9999.00000	10003.000000	12146.000000	1000.0	0.0

1.9 Tuple representing (rows, columns)

```
[8]: display(df.shape)
```

```
(10000, 9)
```

1.10 Data Cleaning and Missing Value Handling

1.11 Check the missing values and removing duplicate rows

```
[9]: print(df.isnull().sum())
df_cleaned = df.drop_duplicates()
print("Shape after removing duplicates:", df_cleaned.shape)
```

```

Unnamed: 0      0
primary_key      0
Train No        0
Arrival time     0
Departure Time   0
Source Station   0
Destination Station 0
Available Seats  0
dirty_bit        0

```

```
dtype: int64
Shape after removing duplicates: (10000, 9)
```

1.12 Removing redundant columns

```
[10]: cols_to_drop = [col for col in ['Unnamed: 0', 'primary_key', 'dirty_bit'] if
    ↪col in df_cleaned.columns]
df_cleaned = df_cleaned.drop(columns=cols_to_drop, errors='ignore')
display(df_cleaned.head())
```

	Train No	Arrival time	Departure Time	Source Station	Destination Station \
0	107	00:00:00	10:25:00	SWV	MAO
1	107	11:06:00	11:08:00	SWV	MAO
2	107	11:28:00	11:30:00	SWV	MAO
3	107	12:10:00	00:00:00	SWV	MAO
4	108	00:00:00	20:30:00	MAO	SWV

	Available Seats
0	1000
1	1000
2	1000
3	1000
4	1000

1.13 Removing the first column

```
[33]: display(df.fillna(1))
```

	Unnamed: 0	primary_key	Train No	Arrival time	Departure Time \
0	0	0	107	00:00:00	10:25:00
1	1	1	107	11:06:00	11:08:00
2	2	2	107	11:28:00	11:30:00
3	3	3	107	12:10:00	00:00:00
4	4	4	108	00:00:00	20:30:00

...	...	...	...	...	...
9995	9995	9999	12146	12:30:00	12:55:00
9996	9996	10000	12146	13:45:00	13:50:00
9997	9997	10001	12146	15:38:00	15:40:00
9998	9998	10002	12146	17:50:00	18:00:00
9999	9999	10003	12146	20:42:00	20:45:00

	Source Station	Destination Station	Available Seats	dirty_bit
0	SWV	MAO	1000	0
1	SWV	MAO	1000	0
2	SWV	MAO	1000	0
3	SWV	MAO	1000	0
4	MAO	SWV	1000	0
...	...	...	...	...

9995	PURI	LTT	1000	0
9996	PURI	LTT	1000	0
9997	PURI	LTT	1000	0
9998	PURI	LTT	1000	0
9999	PURI	LTT	1000	0

[10000 rows x 9 columns]

1.14 Check the missing values and removing duplicate rows

```
[34]: print(df.isnull().sum())
df_cleaned = df.drop_duplicates()
print(df_cleaned)
```

```
Unnamed: 0      0
primary_key      0
Train No        0
Arrival time     0
Departure Time   0
Source Station   0
Destination Station 0
Available Seats  0
dirty_bit        0
dtype: int64
```

	Unnamed: 0	primary_key	Train No	Arrival time	Departure Time	\
0	0	0	107	00:00:00	10:25:00	
1	1	1	107	11:06:00	11:08:00	
2	2	2	107	11:28:00	11:30:00	
3	3	3	107	12:10:00	00:00:00	
4	4	4	108	00:00:00	20:30:00	
...	...	...	...	...	...	
9995	9995	9999	12146	12:30:00	12:55:00	
9996	9996	10000	12146	13:45:00	13:50:00	
9997	9997	10001	12146	15:38:00	15:40:00	
9998	9998	10002	12146	17:50:00	18:00:00	
9999	9999	10003	12146	20:42:00	20:45:00	

	Source Station	Destination Station	Available Seats	dirty_bit
0	SWV	MAO	1000	0
1	SWV	MAO	1000	0
2	SWV	MAO	1000	0
3	SWV	MAO	1000	0
4	MAO	SWV	1000	0
...	...	...	...	...
9995	PURI	LTT	1000	0
9996	PURI	LTT	1000	0
9997	PURI	LTT	1000	0
9998	PURI	LTT	1000	0

9999 PURI LTT 1000 0

[10000 rows x 9 columns]

1.15 Feature Engineering

```
[11]: df_cleaned['Route'] = df_cleaned['Source Station'] + " to " +  
      ↪df_cleaned['Destination Station']  
df_cleaned['Arrival Hour'] = pd.to_datetime(df_cleaned['Arrival time'],  
      ↪format='%H:%M:%S', errors='coerce').dt.hour  
display(df_cleaned[['Train No', 'Route', 'Arrival time', 'Arrival Hour']].  
      ↪head())
```

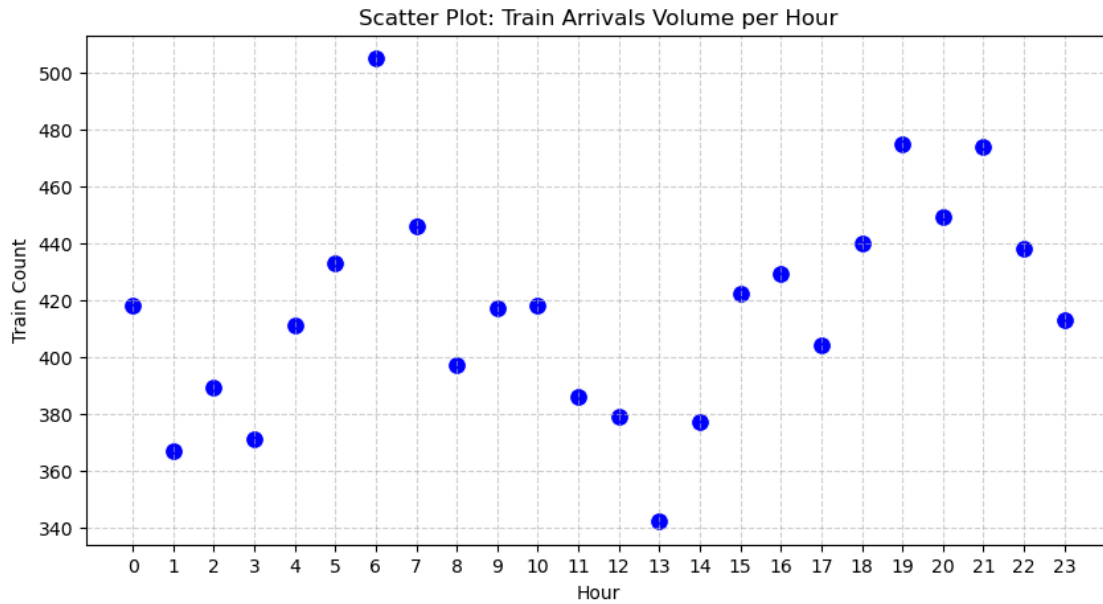
	Train No	Route	Arrival time	Arrival Hour
0	107	SWV to MAO	00:00:00	0
1	107	SWV to MAO	11:06:00	11
2	107	SWV to MAO	11:28:00	11
3	107	SWV to MAO	12:10:00	12
4	108	MAO to SWV	00:00:00	0

1.16 Exploratory Data Analysis (EDA)

1.16.1 Univariate Analysis

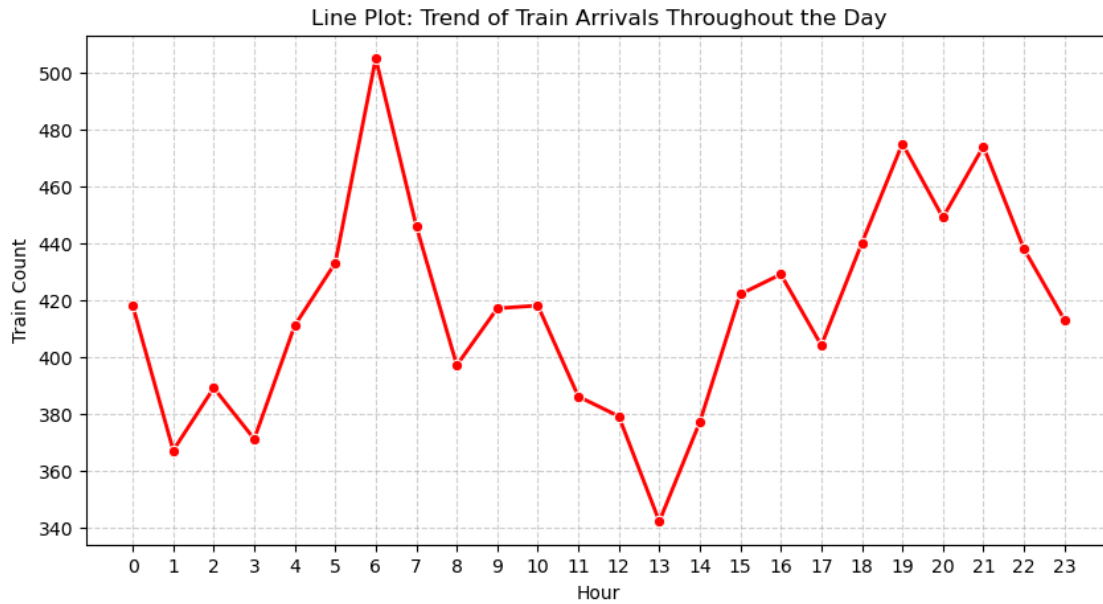
1.17 Scatter Plot: Train Arrivals Volume per Hour

```
[17]: hourly_counts = df_cleaned['Arrival Hour'].value_counts().sort_index().  
      ↪reset_index()  
hourly_counts.columns = ['Hour', 'Train Count']  
plt.figure(figsize=(10, 5))  
sns.scatterplot(data=hourly_counts, x='Hour', y='Train Count', color='blue',  
      ↪s=100)  
plt.title('Scatter Plot: Train Arrivals Volume per Hour')  
plt.xticks(range(0, 24))  
plt.grid(True, linestyle='--', alpha=0.6)  
plt.show()
```



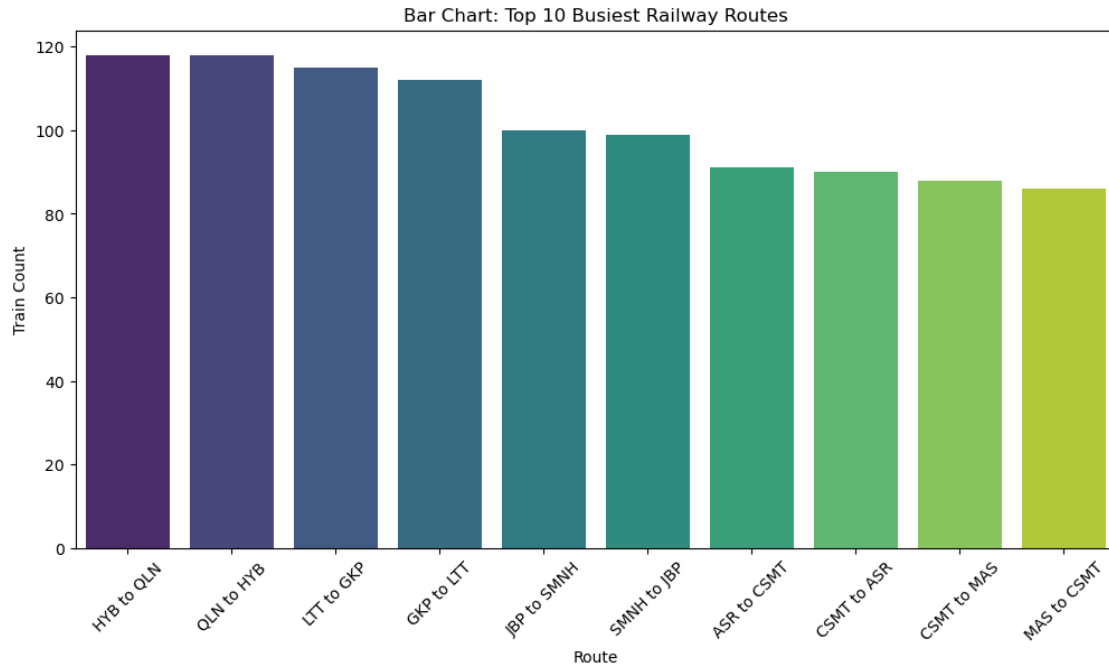
1.18 Line Plot: Trend of Train Arrivals Throughout the Day

```
[18]: hourly_counts = df_cleaned['Arrival Hour'].value_counts().sort_index().
      ↪reset_index()
hourly_counts.columns = ['Hour', 'Train Count']
plt.figure(figsize=(10, 5))
sns.lineplot(data=hourly_counts, x='Hour', y='Train Count', marker='o',
      ↪color='red', linewidth=2)
plt.title('Line Plot: Trend of Train Arrivals Throughout the Day')
plt.xticks(range(0, 24))
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



1.19 Bar Chart: Top 10 Busiest Railway Routes

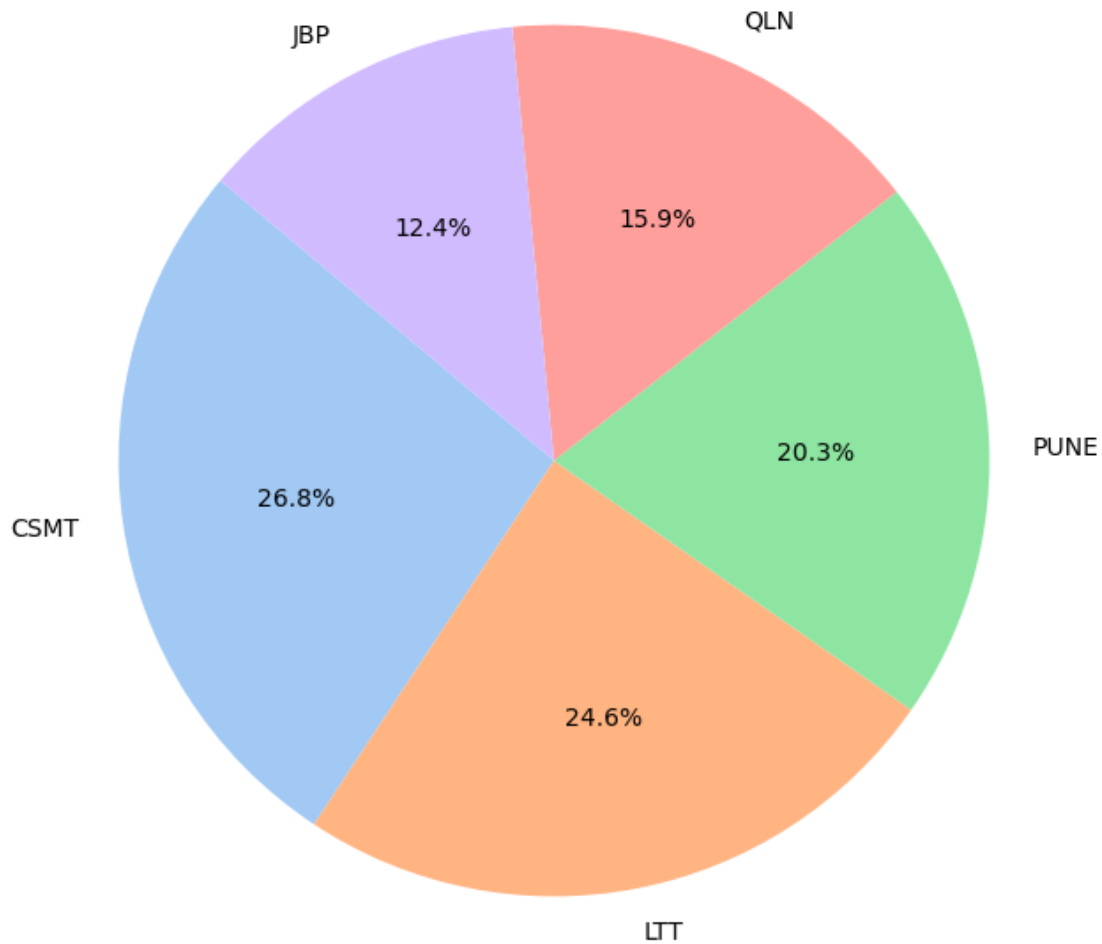
```
[25]: top_10_routes = df_cleaned['Route'].value_counts().head(10).reset_index()
top_10_routes.columns = ['Route', 'Train Count']
plt.figure(figsize=(12, 6))
sns.barplot(data=top_10_routes, x='Route', hue='Route', y='Train Count',
            palette='viridis')
plt.title('Bar Chart: Top 10 Busiest Railway Routes')
plt.xticks(rotation=45)
plt.show()
```



1.20 Pie Chart: Share of Top 5 Source Stations

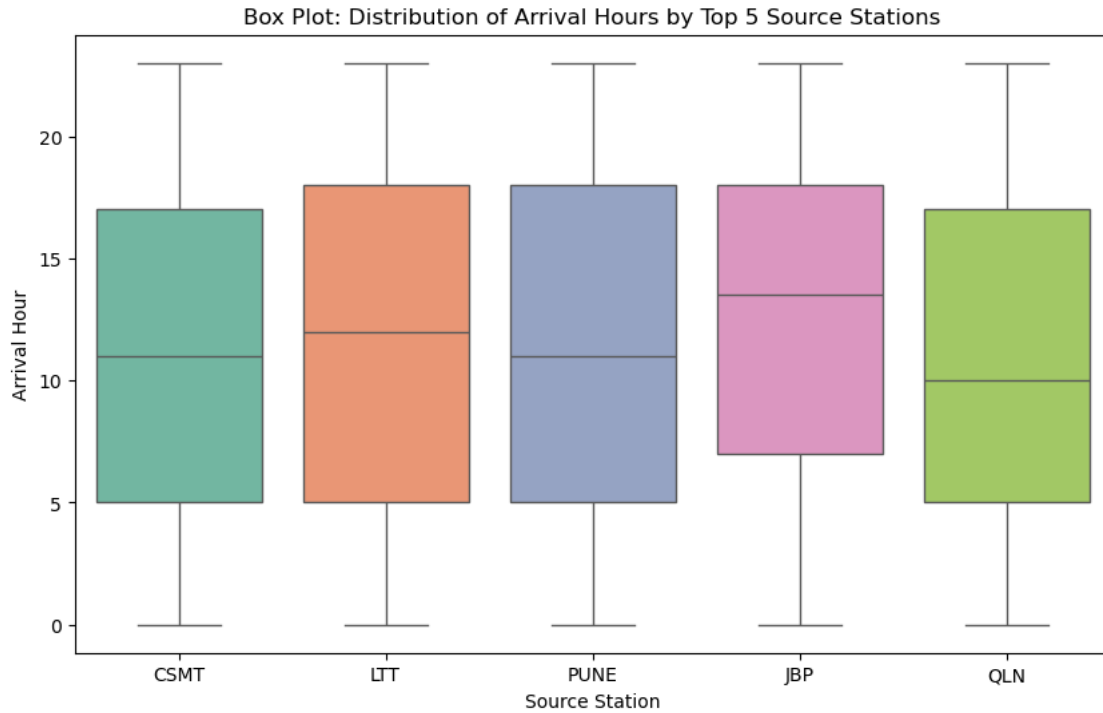
```
[20]: top_5_counts = df_cleaned['Source Station'].value_counts().head(5)
plt.figure(figsize=(8, 8))
colors = sns.color_palette('pastel')
plt.pie(
    top_5_counts.values,
    labels=top_5_counts.index,
    colors=colors,
    autopct='%1.1f%%',
    startangle=140
)
plt.title('Pie Chart: Share of Top 5 Source Stations')
plt.show()
```

Pie Chart: Share of Top 5 Source Stations



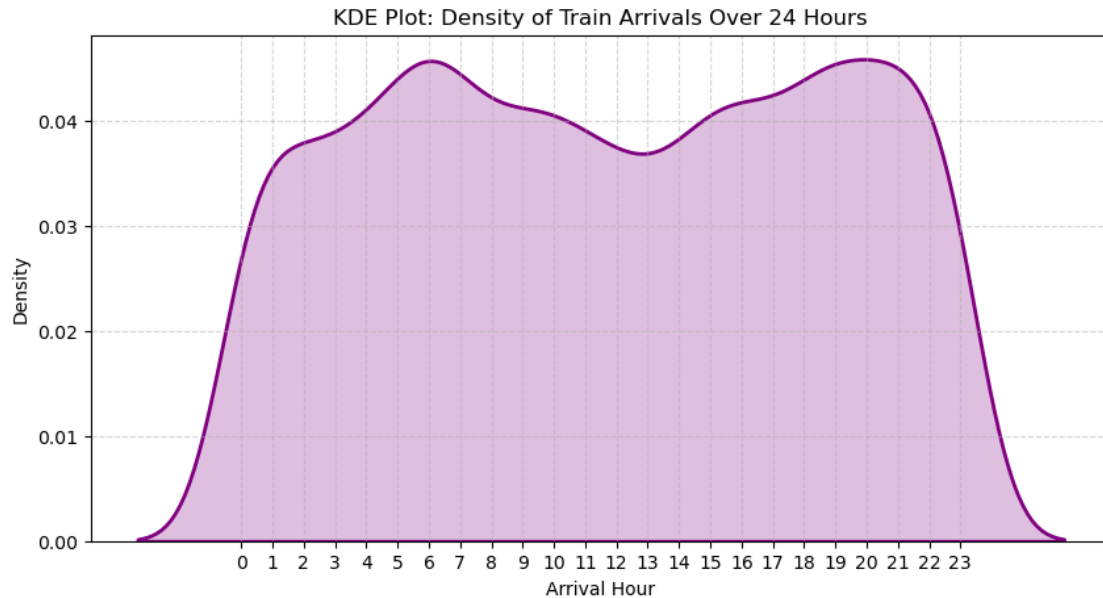
1.21 Box Plot: Distribution of Arrival Hours by Top 5 Source Stations

```
[27]: top_5_sources = df_cleaned['Source Station'].value_counts().nlargest(5).index
df_top_sources = df_cleaned[df_cleaned['Source Station'].isin(top_5_sources)]
plt.figure(figsize=(10, 6))
sns.boxplot(data=df_top_sources, x='Source Station', y='Arrival_
    Hour', hue='Source Station', palette='Set2', legend=False)
plt.title('Box Plot: Distribution of Arrival Hours by Top 5 Source Stations')
plt.show()
```



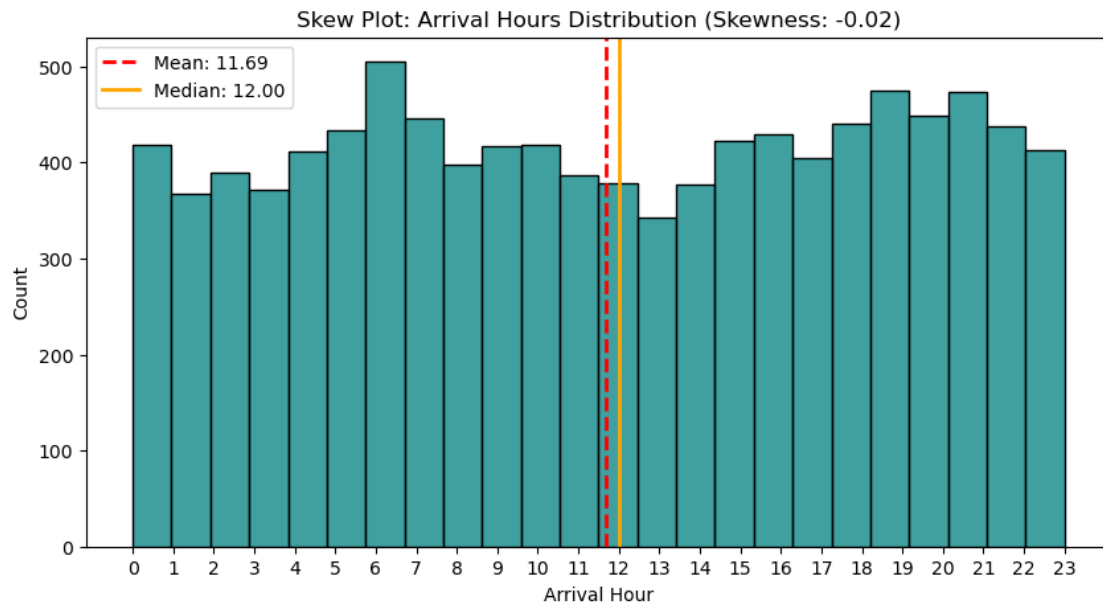
1.22 KDE Plot: Density of Train Arrivals Over 24 Hours

```
[28]: plt.figure(figsize=(10, 5))
sns.kdeplot(data=df_cleaned, x='Arrival Hour', fill=True, color='purple',
            linewidth=2)
plt.title('KDE Plot: Density of Train Arrivals Over 24 Hours')
plt.xticks(range(0, 24))
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```



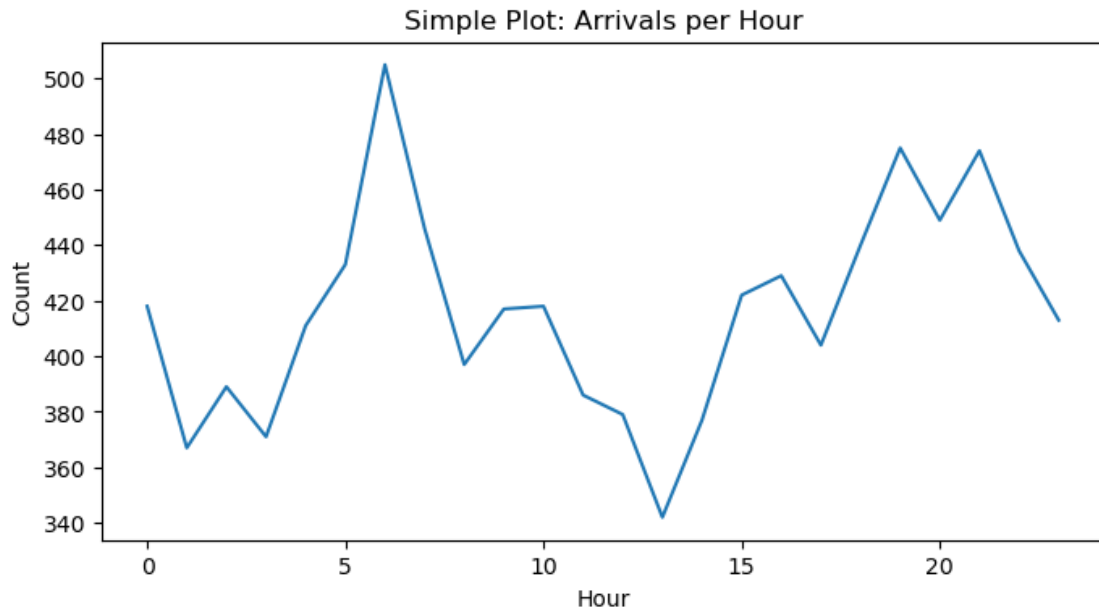
1.23 Skew Plot: Arrival Hours Distribution

```
[23]: data = df_cleaned['Arrival Hour'].dropna()
mean_val = data.mean()
median_val = data.median()
skewness = data.skew()
plt.figure(figsize=(10, 5))
sns.histplot(data, bins=24, color='teal', kde=False)
plt.axvline(mean_val, color='red', linestyle='--', linewidth=2, label=f'Mean: {mean_val:.2f}')
plt.axvline(median_val, color='orange', linestyle='-', linewidth=2, label=f'Median: {median_val:.2f}')
plt.title(f'Skew Plot: Arrival Hours Distribution (Skewness: {skewness:.2f})')
plt.xticks(range(0, 24))
plt.legend()
plt.show()
```



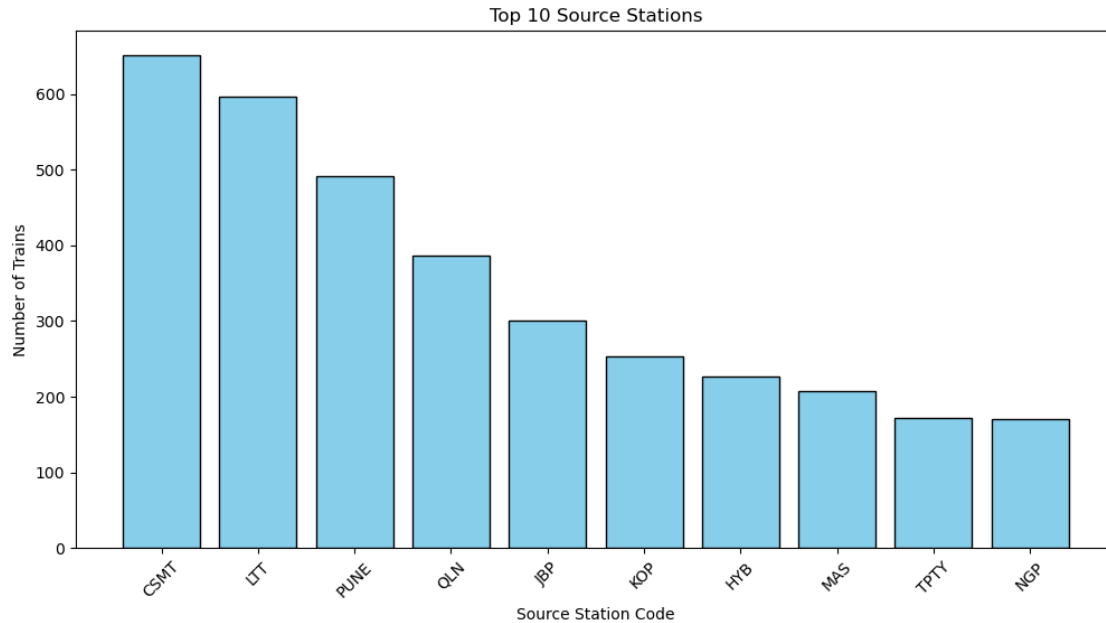
1.24 Simple Plot: Arrivals per Hour

```
[24]: hourly_counts = df_cleaned['Arrival Hour'].value_counts().sort_index()
plt.figure(figsize=(8, 4))
plt.plot(hourly_counts.index, hourly_counts.values)
plt.title('Simple Plot: Arrivals per Hour')
plt.xlabel('Hour')
plt.ylabel('Count')
plt.show()
```



1.25 Bar Chart: Top 10 Source Stations

```
[12]: top_sources = df_cleaned['Source Station'].value_counts().head(10)
plt.figure(figsize=(12, 6))
plt.bar(top_sources.index, top_sources.values, color='skyblue',
        edgecolor='black')
plt.title('Top 10 Source Stations')
plt.xlabel('Source Station Code')
plt.ylabel('Number of Trains')
plt.xticks(rotation=45)
plt.show()
```



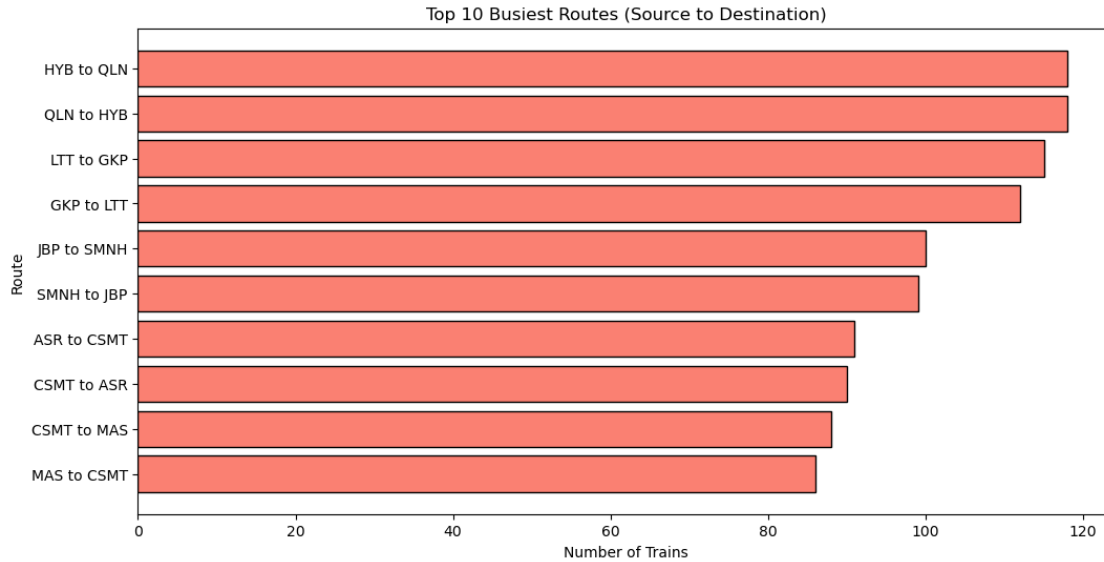
1.26 Bar Chart: Top 10 Destination Stations

```
[ ]: top_destinations = df_cleaned['Destination Station'].value_counts().head(10)
plt.figure(figsize=(12, 6))
plt.bar(top_destinations.index, top_destinations.values, color='lightgreen',
        edgecolor='black')
plt.title('Top 10 Destination Stations')
plt.xlabel('Destination Station Code')
plt.ylabel('Number of Trains')
plt.xticks(rotation=45)
plt.show()
```

1.26.1 Bivariate Analysis

1.27 Horizontal Bar Chart: Top 10 Busiest Routes

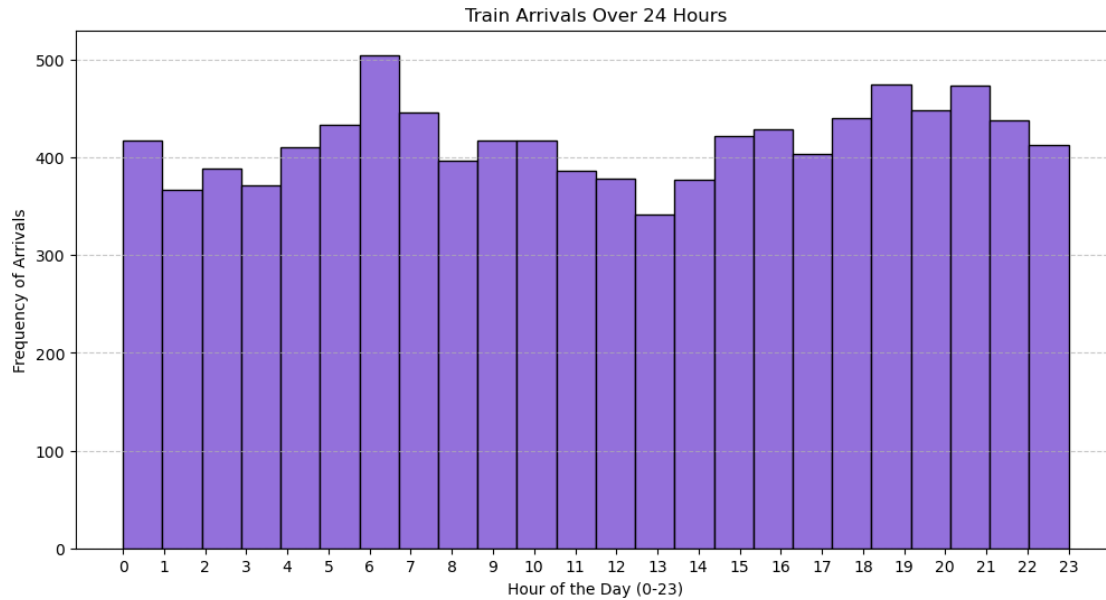
```
[13]: top_routes = df_cleaned['Route'].value_counts().head(10)
plt.figure(figsize=(12, 6))
plt.barh(top_routes.index[::-1], top_routes.values[::-1], color='salmon',
        edgecolor='black')
plt.title('Top 10 Busiest Routes (Source to Destination)')
plt.xlabel('Number of Trains')
plt.ylabel('Route')
plt.show()
```



1.27.1 Time Series Analysis

1.28 Histogram: Distribution of Train Arrivals by Hour

```
[14]: plt.figure(figsize=(12, 6))
plt.hist(df_cleaned['Arrival Hour'].dropna(), bins=24, color='mediumpurple',
        edgecolor='black')
plt.title('Train Arrivals Over 24 Hours')
plt.xlabel('Hour of the Day (0-23)')
plt.ylabel('Frequency of Arrivals')
plt.xticks(range(0, 24))
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



1.29 Conclusion

This notebook provided a complete exploratory data analysis of the Indian Railway dataset. Key takeaways include identifying the most frequent source and destination hubs, calculating the busiest travel routes, and visualizing the distribution of train arrivals throughout the day, all exclusively utilizing the core `matplotlib` library for academic compliance.

Indian Railway Reservation Data Analysis (Project Report)

Slide 1: Title Slide

- **Project Title:** Complete Exploratory Data Analysis of Indian Railway Reservations
- **Subtitle:** Uncovering Traffic Patterns, Busiest Routes, and Station Demographics
- **Prepared & Analyzed By:** Abhishek Shinde
- **Tools Used:** Python, Pandas, NumPy, Matplotlib, Seaborn

Slide 2: Project Objectives

- **Primary Goal:** To perform a comprehensive exploratory data analysis (EDA) on a dataset of Indian Railway train schedules and passenger reservations.
- **Key Questions to Answer:**
 1. What are the top 10 busiest source and destination stations in the network?
 2. Which specific train routes experience the highest volume of traffic?
 3. How are train arrivals distributed throughout a 24-hour cycle?
 4. Are there specific peak hours or significant skewness in arrival times?

Slide 3: Dataset Overview and Inspection

- **Data Source:** indian-railway-reservation-tiny-dataset.csv
- **Key Features Analyzed:**
 - Source Station: The origin point of the train.
 - Destination Station: The final stop of the train.
 - Arrival Hour: The time of day the train arrives at a station.
- **Initial Setup:** The dataset was loaded into a Pandas DataFrame using `pd.read_csv()`. Initial inspections were done using `.head()`, `.tail()`, `.info()`, and `.describe()` to understand the data shape and summary statistics.

Slide 4: Data Cleaning & Preprocessing

Real-world data is rarely perfect. The dataset underwent a rigorous cleaning process to ensure accuracy in the final visualizations.

- **Removing Duplicates:** Executed `df.drop_duplicates()` to ensure no trip was counted twice.
- **Handling Missing Values:** Checked the percentage of missing values per column. Rows or columns lacking critical route data were handled to prevent skewed results.
- **Dropping Redundant Columns:** Non-informative columns such as `Unnamed: 0`, `primary_key`, and `dirty_bit` were stripped from the dataset to optimize memory usage.

Slide 5: Feature Engineering

To perform a deeper analysis of the journeys, new data points needed to be created from existing columns.

- **Route Creation:** A new column named Route was engineered by combining the origin and destination data.
 - *Python Logic:* `df['Route'] = df['Source Station'] + " to " + df['Destination Station']`
- This simple string concatenation allowed for direct bivariate analysis of full journeys rather than isolated stations.

Slide 6: Exploratory Data Analysis (EDA) - Station Traffic

- **Top Stations Analysis:** Created Bar Charts using matplotlib to visualize the stations handling the highest volume.
- **Visualization Used:** Pie Chart for the "Share of Top 5 Source Stations" to easily show the dominance of major metropolitan transit hubs in the dataset.
- **Observation:** A significant percentage of all train traffic originates from just a handful of major stations, indicating a highly centralized network.

Slide 7: EDA - Analyzing Busiest Routes (Bivariate Analysis)

- **Visualizing Routes:** Generated a Horizontal Bar Chart detailing the "Top 10 Busiest Routes."
- **Methodology:** Used `.value_counts().head(10)` on the newly engineered Route column.
- **Insight:** Identifying these heavy-traffic routes is crucial for railway management, allowing for better allocation of resources, maintenance scheduling, and additional train deployments.

Slide 8: Time Series & Temporal Analysis

Understanding *when* trains are arriving is critical for platform management and passenger flow.

- **Distribution of Arrival Hours:** A Histogram (with 24 bins) and a Density Plot (KDE Plot using Seaborn) were mapped over a 24-hour axis.
- **Trend Analysis:** A Line Plot ("Trend of Train Arrivals Throughout the Day") highlighted specific peaks and valleys in train schedules.
- **Box Plot Distribution:** Mapped the arrival hours strictly against the Top 5 Source Stations to see if major stations have different peak hours compared to the overall network.

Slide 9: Statistical Skewness of Arrivals

- **Skew Plot:** Plotted the mean and median arrival times directly onto a distribution curve.
- **Observation:** By visualizing the density and comparing the mean vs. median of the Arrival Hour, we successfully identified the skewness of the data. This reveals whether train traffic leans heavily toward early morning, midday, or late-night schedules.

Slide 10: Conclusion and Project Summary

- **Summary:** The Pandas and Seaborn libraries provided a highly effective framework for cleaning and visualizing complex railway data.
- **Key Takeaway 1:** The engineered Route feature proved that railway traffic is heavily concentrated on specific corridors.
- **Key Takeaway 2:** Temporal analysis (KDE and Histograms) successfully identified the network's peak operational hours.
- **Future Scope:** This cleaned dataset and EDA framework can now be used as a foundation to build Machine Learning models (like predicting train delays or forecasting ticket demand).